

大语言模型辅助的分层强化学习训练

Large language model assisted hierarchical reinforcement learning training

Qianxi Li^{a,b}, Bao Pang^{a,b,*}, Yong Song^{a,b}, Hongze Fu^a, Qingyang Xu^a, Xianfeng Yuan^a, Xiaolong Xu^a, Chengjin Zhang^a

^a 山东大学机械、电气与信息工程学院, 中国山东威海 264209

^b 山东省智能电子封装测试与应用重点实验室, 中国山东威海 264209

翻译范围: 原文全文 (第 1-21 页)。LLM、RL、HRL、VLM、PPO、RRT* 等通用缩写按原文保留; 参考文献的英文题名与出版信息保留, 以便准确检索。

研究亮点

- 提出 LLMHRL: 一种由语言模型辅助的分层强化学习方法。
- 高层策略接受语言模型指导, 从而提升性能。
- 将高层策略分解为技能与目标, 以缩小动作空间。

关键词

强化学习; 大语言模型; 序列决策

摘要

传统强化学习 (RL) 难以解决复杂的长序列决策任务, 尤其是在环境奖励稀疏时。大语言模型 (LLM) 凭借强大的推理能力, 能够在长序列决策任务中取得良好表现。然而, 尽管 LLM 拥有大量通用知识, 基于 LLM 的智能体在解决特定目标问题时仍缺乏专业能力。考虑到强化学习模型的规模小于 LLM, 并且可以针对特定任务进行专门训练以获得良好性能, 本文提出一种由大语言模型辅助的分层强化学习框架, 称为 LLMHRL。

在该框架中, LLM 充当教师智能体, 引导分层强化学习高层策略的探索。低层策略由一个可供选择的策略库构成; 智能体依据高层策略选定的低层策略执行具体动作。此外, 为缩小高层策略的动作空间, 本文将高层动作分解为技能选项和目标选项, 并将二者组合成高层策略。本文在三个环境中, 以公开任务和自建的更高难度任务将 LLMHRL 与基线方法进行比较: MiniGrid 中的钥匙-门匹配、ManiSkill 中的桌面分拣, 以及真实世界场景。结果表明, LLMHRL 在成功率、收敛速度和平均回报方面优于现有方法。

1. 引言

现代最优控制的目标之一, 是制定能够完成长序列任务的策略。这类任务要求对环境有准确理解, 并具备强大的推理能力 [1]。RL 在求解序列决策问题方面已经取得了巨大成功。

最优策略通过智能体与环境交互来学习, 从而降低对先验知识的需求。

然而, 面对长序列决策任务时, 传统 RL 的性能会显著下降。由于这类任务具有庞大的状态空间和动作空间, 传统 RL 在求解时的表现明显变差。多数研究通过设计奖励函数来弥补探索不足 [2]。但期望行为难以量化, 而稠密且准确的奖励又需要将任务分解为子目标 [3], 或设置逐步达成的目标, 因此高性能奖励函数很难设计。

分层强化学习 (HRL) 通过缩短任务时域来提高任务成功率, 并已被证明尤其适用于大动作空间 [4]。HRL 采用分而治之的方法处理问题: 高层策略在抽象状态空间上运行, 由外部奖励引导; 低层策略则遵循独立的执行策略, 并在成功执行后获得与全局奖励相互隔离的独立奖励。

HRL 的优越表现可归因于更有效的奖励信用分配和更高效的探索。通过对动作序列进行抽象, HRL 能够更快学习价值函数, 因为奖励可以更准确地向序列前端传播。Driess 等人 [5] 指出, HRL 以具有语义意义、并在时间上延展的方式进行探索, 而传统 RL 容易对邻近状态进行过度探索。

* 通信作者: 山东大学机械、电气与信息工程学院, 中国山东威海 264209。

电子邮箱: liqianxi@mail.sdu.edu.cn; pangbao@sdu.edu.cn; songyong@sdu.edu.cn; fuhongze@mail.sdu.edu.cn; qingyangxu@sdu.edu.cn; yuanxianfeng@sdu.edu.cn; xuxiaolong@sdu.edu.cn; cjzhang@sdu.edu.cn

DOI: 10.1016/j.ins.2025.122688。收稿: 2025-04-11; 修回: 2025-09-12; 接受: 2025-09-12; 在线发表: 2025-09-17。

© 2025 Elsevier Inc. 保留所有权利, 包括与文本和数据挖掘、AI 训练及类似技术有关的权利。

另一种改进 RL 的途径是采用基于 LLM 的指导。LLM 已在机器人、故障诊断、网络安全 [6] 等领域展现潜力，其推理能力与广泛的通用知识可显著提升 RL 性能。尽管尚无将 LLM 与 RL 结合的统一方法，研究者已经探索了多种策略：利用 LLM 预处理状态、塑造奖励、直接决策以提高采样效率 [7]、生成世界动力学模型，或充当专家指导者 [8,9]。例如，LLM 可将历史轨迹 [10] 或自然语言指令 [11] 转换成任务专用语言，从而提高训练效率；也可以充当隐式奖励 [12] 或显式奖励 [13]。

在现有 HRL 方法中，低层策略可以由专家预先定义，也可以来自先验知识；但 HRL 的高层策略通常仍需从零开始学习，这会限制其性能。本文提出一种新方法，利用 LLM 先验知识来辅助 HRL 的高层策略学习。

由于 LLM 只能处理文本输入，本文首先使用视觉语言模型 (VLM) 将环境图像转换为文字描述，再将这些描述与任务信息一同输入 LLM，以生成可解释的高层动作指令，例如“拿起锤子”或“前往蓝色钥匙”。低层策略由多个预定义的技能模块组成，它们根据高层指令执行相应动作。在训练期间，LLM 不直接与环境交互，而是作为外部奖励信号来指导 HRL；这种指导的权重会随训练逐渐降低。HRL 由向 LLM 学习逐步过渡到通过环境交互学习，最终超越 LLM 的表现。

本文还提出一种新的高层策略动作嵌入方法，将高层动作分解为技能与目标。目标表示高层策略所选择的交互对象（如门、钥匙等），会随环境变化；技能表示与目标交互的方式（如拿取、打开等基本动作），在不同环境中保持一致。

综上，LLMHRL 的主要贡献如下：

- (1) 为提升 HRL 在长序列任务中的性能，提出一种由 LLM 辅助训练 HRL 的方法。
- (2) 针对长序列任务中高层策略动作空间庞大的问题，提出将高层动作分解为“技能 + 目标”组合的方法。

2. 相关工作

本文利用 LLM 的推理能力和先验知识，解决稀疏奖励环境下长序列决策任务所面临的挑战。将 LLM 应用于这类任务仍存在许多问题，例如 LLM 的实时响应、复杂任务中的推理能力，以及 LLM 对环境理解。已有相关研究针对这些问题提出了可落地的实现方法。

2.1 LLM 的实时、低成本实现

在处理长序列决策任务时，频繁查询 LLM 可能导致成本过高（若使用商业模型）或推理时间过长。在 Yu 的方法 [10] 中，智能体每执行一个动作都必须咨询 LLM，而 LLM 需要数秒才能响应，因此无法实现实时控制。为解决这一问题，研究者采取了多种方法。Bang 的方法 [11] 开发了 GPTCache，用于查询语义响应，使响应速度提高 2-10 倍。另一些方法仅在训练阶段使用 LLM，再通过 RL 训练最优策略，例如设计奖励 [14]、提升推理能力 [13]。Miao [15] 探索将 LLM 融入机器人操作，提出一种分层知识表示框架，通过多源知识更新和基于嵌入的推理技术增强任务理解。

2.2 提升 LLM 的推理能力

若要让 LLM 直接处理长序列任务，就需要提升其推理能力。正如 Valmeekam [16] 和 Lou [17] 所指出的，LLM 会以自回归方式本能地产生推理轨迹，形成一条假设性的语义逻辑链。然而，

人类拥有内部世界模型，会反复探索不同选项并评估可能结果。换言之，LLM 需要适当的奖励机制、世界模型，以及对探索与利用的平衡。Huang 的方法 [18] 提出类似对话的内在独白，以充分观察环境并利用环境反馈。思维链和加入示例，是拆解复杂问题并提供可解释窗口的常用方法 [19]。文献 [20] 表明，仅加入“让我们一步一步思考”也能提升 LLM 的零样本推理性能。另一种广泛使用的方法是基于人类反馈的强化学习，例如使用人类偏好数据的 MaxMin-RLHF [21]。

2.3 输入输出与环境对齐

使输入和输出与环境对齐具有挑战性，因为 LLM 必须在输入有限的情况下准确感知环境信息。尤其是，LLM 可能生成环境无法执行的错误动作，或者无法准确理解环境动力学，特别是隐含的相互依赖关系。

RL 可用于输出层，以弥补 LLM 的对齐不足，因为算法能够从具体环境中学习。在 Tan 的方法 [22] 中，演员与评论家共享同一个 LLM，并通过 PPO 更新低秩适配器。Carta [23] 输出宽泛的高层指导，再用 PPO 进行微调，以限制不对齐。Szot [24] 在生成端引入 MLP 层，并将动作与价值拆分以提高精度；同时还通过编码器引入嵌入后的任务指令和视觉观测。

2.4 与现有 LLM-RL 框架的比较

表 1 与现有 LLM-RL 框架的比较

方法	模型结构	适用任务序列长度	推理成本
EUREKA [25]	RL	短	高
Text2Reward [26]	RL	短	高
LLMTeach [27]	RL	中	低
LLM4PG [28]	RL	中	低
SayCan [29]	HRL	长	高
SayCanPay [30]	HRL	长	高
GLIDER [31]	HRL	长	高
LLMHRL (本文)	HRL	长	低

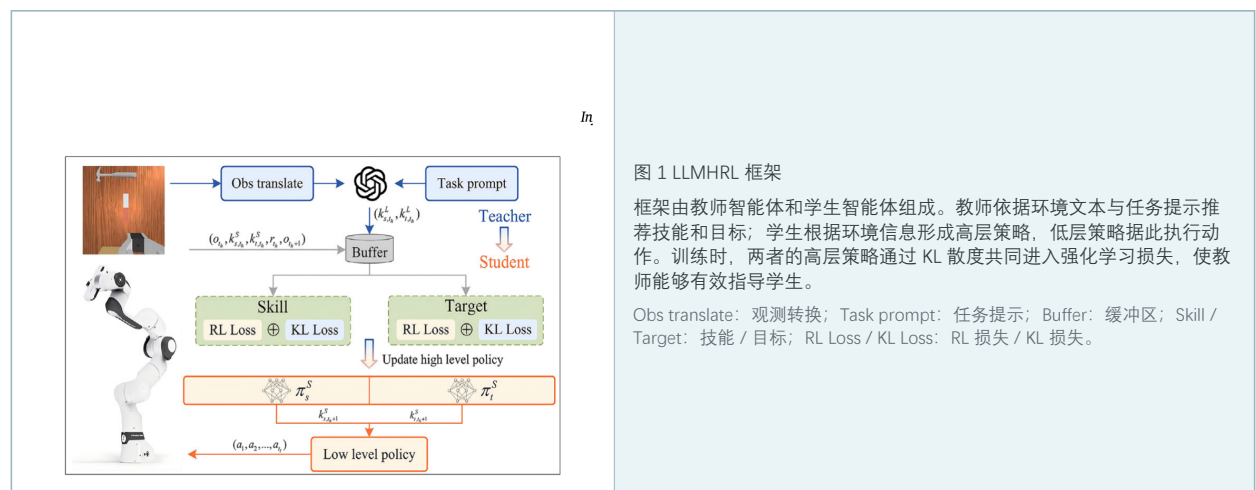
表 1 从模型结构、适用任务序列长度和推理成本三个方面比较了本文方法与现有方法。早期 LLM-RL 框架使用 LLM 设计 RL 奖励函数，从而避免复杂的人工奖励设计 [25,26]。这些方法引入反馈机制，迭代优化奖励设计并提升 RL 性能。然而，奖励设计的迭代优化需要反复重新训练 RL 智能体并评估性能，因而带来显著的时间和计算开销。随后，LMTeach [27] 与 LLM4PG [28] 提出将 LLM 策略直接蒸馏到 RL 中：LLM 给出建议的智能体行为或轨迹并存入经验回放缓冲区，从而提高 RL 的样本效率。这些方法使 LLM 对 RL 的指导更直接，但在包含多个子任务的长序列任务中仍面临困难，因为回放缓冲区中有效轨迹数量有限，阻碍 RL 学到有效策略。

长序列决策任务会加剧探索-利用困境，因此对强化学习尤其困难。为提升 RL 在长时域决策任务中的性能，LLM-HRL 框架使用高层策略选择预定义技能，从而缩短动作序列并提高 RL 性能。例如，GLIDER [31] 通过 RL 微调一个 LLM，使其充当高层策略；SayCan [29] 与 SayCanPay [30] 则训练评估模型，帮助 LLM 选择任务技能。然而，这些方法在部署阶段始终需要在线调用 LLM 推理，导致计算开销大、模型推理慢，因而难以部署到算力有限且实时性要求高的机器人设备上。

本文提出的 LLMHRL 保留 HRL 框架，并将一个由 LLM 指导训练的 RL 智能体作为高层策略。对于长序列任务，LLMHRL 在训练早期借助 LLM 指导快速提高任务成功率，后期再通过与环境交互学习进一步提升性能。此外，在部署阶段，LLMHRL 不再依赖 LLM 推理，而只依靠 HRL 输出动作，从而降低计算与时间成本。

在不改变 RL 范式的前提下使用 LLM 指导 RL，是一种广泛采用的策略。EUREKA [25] 与 Text2Reward [26] 使用 LLM 辅助构造稠密奖励；但稠密奖励可能不是最优的指导方式，因为它们只评估学生智能体的即时动作。LLMTeach [27] 通过给出建议动作提供更明确的指导，LLM4PG [28] 则从长期角度评估学生动作。分层框架通过预定义技能生成低层动作，缩小动作空间，并重点增强高层决策。SayCan [29]、SayCanPlay [30] 和 GLIDER [31] 在部署时均直接使用 LLM 作出高层决策。HKR [15] 使用 LLM 优化用于高层决策的静态知识图谱。相比之下，LLMHRL 使用动态神经网络进行高层决策，并结合 LLM 建议与环境反馈来增强鲁棒性。

框架概览



3. 方法

本节首先形式化研究问题，随后介绍 LLMHRL 框架。

3.1 问题形式化

序列决策问题通常建模为马尔可夫决策过程, 记为 $M = (O, A, T, r, \gamma)$ 。其中, O 表示观测空间, A 表示动作空间, $T: S \times A \rightarrow P(O)$ 表示转移概率, r 为奖励函数, γ 为折扣因子。该过程旨在使 $\sum_t \gamma^t r(s_t, a_t)$ 最大的最优策略, 其中 t 为时间步。

在奖励稀疏的长序列决策任务中, 强化学习很难学到最优策略。引入先验知识可以提高样本效率并加快训练。基于选项的 HRL 将动作划分为高层动作和低层动作; 每个高层动作 (即选项) 由现有知识或策略给出的一系列低层动作组成, 智能体学习高层动作策略以完成整体任务。

本文采用基于选项的 HRL 来描述问题, 引入可跨越多个时间步的时间延展动作。选项 W 是三元组 (L_w, π_w, β_w) , 分别表示可启动该选项的状态集合、选项策略和终止函数。在观测 o 下, 高层策略选择满足 $o \in L_w$ 的选项策略 π_w , 并将若干时间步的动作控制权交给该选项。当终止条件 β_w 满足时, 选项结束并将控制权交还高层策略, 由其选择下一选项。

3.2 框架概述

如图 1 所示, 框架包含教师与学生。每个高层时间步 t_h , 双方都接收观测 o_{t_h} 。学生据此采取高层动作, 低层策略在其指导下生成动作序列 $a_1, a_2, \dots, a_{t_h}$ 。当低层策略满足终止条件 β_w 后, 观测 o_{t_h} 成为下一次高层决策的观测 $o_{t_{h+1}}$ 。教师根据 o_{t_h} 给出指导策略, 该策略与环境奖励共同用于训练学生的高层策略。

3.3 教师智能体

如图 2 所示, 本文定义高层选项集合 K_s 与 K_t , 分别表示智能体可选择的任务技能和任务目标。输入 LLM 的提示文本 $p \in P$ 由两部分构成: p_m 描述当前任务, p_o 描述当前环境观测 o 。环境观测可由 GPT-4V、Qwen-VL 等视觉语言模型转换为文本。随后, LLM 根据 p 生成高层策略 a^L 。硬编码的动作分解模块再将其分解为技能选项 k^L 与目标选项 k^L , 用于指导学生。二者均采用 one-hot 编码。

由于 LLM 不直接与环境交互, 无法确认生成的 a^L 是否是当前观测下的最佳高层决策。本文使用同一提示查询 LLM 共 n 次, 分别记录技能和目标的选项分布, 并形成教师策略 π_s^L 与 π_t^L 。

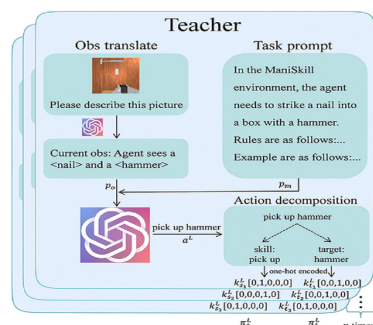
**Fig. 2.** Teacher pipeline

图2 教师智能体流程。图中展示了观测转换、任务提示、LLM 决策以及动作分解过程。

$$\pi_s^L = (1/n) \sum_{i=1}^n k_{s,i}^L \quad (1)$$

$$\pi_t^L = (1/n) \sum_{i=1}^n k_{t,i}^L \quad (2)$$

其中, $k_{s,i}^L$ 表示第 i 次查询 LLM 得到的技能选项分布向量, $k_{t,i}^L$ 表示相应的目标选项分布向量。 π_s^L 与 π_t^L 将在学生网络训练期间提供指导。

3.4 学生智能体

(1) 高层策略: 学生采用基于选项的 HRL, 如图 3 所示。高层策略包含技能模块与目标模块: 技能模块从 K_s 中选择技能, 目标模块从 K_t 中选择当前技能的作用目标。高层决策由两类选项组合而成, 例如“拿起 + 锤子”。当技能多、环境中的可交互对象多时, 这种组合能够显著缩小高层决策的动作空间。

具体而言, 本文使用一个标准卷积层感知环境, 再通过两个独立的全连接层分别输出技能和目标; 两个输出层均采用 softmax, 得到学生的高层策略 π_{ζ}^S 与 π_{τ}^S 。它们分别产生选项分布向量 k_{ζ}^S 与 k_{τ}^S , 组合成高层动作并传递给底层策略。

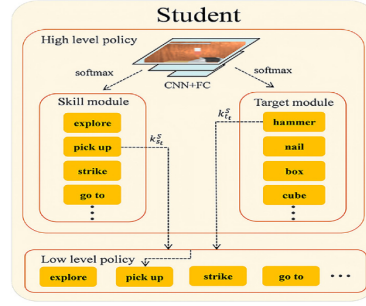


Fig. 3. Student pipeline.

图 3 学生智能体流程。CNN 与全连接层分别生成技能模块和目标模块的选项分布，再组合为高层动作；低层策略执行具体技能。

(2) 低层策略：低层策略根据高层策略生成具体动作并与环境交互。本文采用多个依据人类专家知识硬编码的策略选项，也可替换为预训练的强化学习模块。高层策略被传入对应的低层选项后，低层策略 π_t 根据高层目标与环境生成动作序列 $a_1, a_2, \dots, a_{t_1}$ ，直至满足终止条件 β_w 或超过最大步数 T_1 。每个低层动作都会获得观测和环境奖励；从 r_1 到 r_{t_1} 的奖励和，视为该高层动作获得的奖励 $r_{t,h}$ ，并用于训练学生。

3.5 训练模块

学生的高层策略 π_s^S 与 π_t^S 通过最小化以下损失函数进行训练：

$$L(s,t) = L_{RL}(s) + L_{RL}(t) + \lambda_1 [E_{o \sim \pi_s^L} H(\pi_s^L(o) \parallel \pi_s^S(o)) + E_{o \sim \pi_t^L} H(\pi_t^L(o) \parallel \pi_t^S(o))] \quad (3)$$

$$H(\pi_t^L(o) \parallel \pi_t^S(o)) = D_{KL}(\pi_t^L(o) \parallel \pi_t^S(o)) = \pi_t^L(o) \log[\pi_t^L(o) / \pi_t^S(o)] \quad (4)$$

$$H(\pi_s^L(o) \parallel \pi_s^S(o)) = D_{KL}(\pi_s^L(o) \parallel \pi_s^S(o)) = \pi_s^L(o) \log[\pi_s^L(o) / \pi_s^S(o)] \quad (5)$$

$L_{RL}(s)$ 与 $L_{RL}(t)$ 分别是 π_s^S 和 π_t^S 使用的强化学习损失。本文采用近端策略优化 (PPO)，通过最大化智能体的期望回报从环境中学习。两项 H 衡量教师推荐的高层策略与学生给出的高层策略之间的差异，可使用 KL 散度或余弦相似度计算。

本文引入退火参数 λ 控制教师对学生的指导程度：

$$\lambda_i = \{\lambda_0 - (\lambda_0/i_1) \cdot i, \text{ 若 } i < i_1; 0, \text{ 其他情况} \} \quad (6)$$

$$\lambda_0 = [L_{RL}(s) + L_{RL}(t)] / [E_{o \sim \pi_s^L} H(\pi_s^L \parallel \pi_s^S) + E_{o \sim \pi_t^L} H(\pi_t^L \parallel \pi_t^S)] \quad (7)$$

训练开始时，为平衡 KL 散度损失与 RL 损失、避免二者差距过大，本文用式 (7) 设置 λ_0 。随着训练步数 i 增加， λ 以 λ_0/i_1 的速率线性减小，并在 i_1 步后降为 0。此后，学生完全通过与环境交互学习，进一步提高其针对特定任务的鲁棒性。完整流程见算法 1。

算法 1 学生学习过程

初始化策略 π_{s_skill} 、 π_{s_target} 的参数 s 、 t ；预训练低层策略选项 $\{p_{i_low}\}$

for $i = 1, 2, \dots, T$:

置数据缓冲区 $D = \{\}$

for $j = 1 \dots \text{batch_size}$:

for $t_high = 1 \dots T_high$:

beta = False; $t_low = 0$

$kS_skill = \pi_{s_skill}(o_{t_high})$; $kS_target = \pi_{s_target}(o_{t_high})$

while $t_low < T_low$:

a_{t_low} , beta = $\pi_{i_low}(o_{t_low}, kS_skill, kS_target)$

执行动作 a_{t_low} ，观测 r_{t_low} 与下一观测； $t_low += 1$

if beta == True:

$r_high = \text{sum}(r_low)$; $o_next = o_{t_low}$; break

将 $(o_{t_high}, kS_skill, kS_target, r_high, o_next)$ 存入 D

对 D 中每条转移:

根据环境观测生成教师提示 p

按式 (1) (2) 得到教师指导策略 π_{i_skill} 、 π_{i_target}

每个梯度下降步骤按式 (3) 更新 $L(s,t)$

4. 实验

本节通过多组实验评估 LLMHRL。结果表明，与其他基线方法相比，LLMHRL 具有明显优势。

4.1 实验环境

MiniGrid: MiniGrid 提供网格化视觉环境，可自由设置网格大小以及墙、门、钥匙等交互对象。对象状态可通过交互改变，例如将关闭的门打开。智能体需要导航至指定对象并完成一系列交互，以完成“将所有球放入盒子”等任务。动作空间为离散动作，包括前进、左转、右转、打开、关闭和停止。观测空间设为 3×3 的局部可观测区域，并带有记忆功能，可记录已发现的网格或目标。

ManiSkill: ManiSkill3 是面向具身 AI 的 GPU 并行机器人仿真平台，用于加速可泛化操作技能的开发。它支持多种接触密集型任务，包括灵巧物体重定向、工具绘图、移动操作和人形控制，可用于艺术家设计场景与现实世界数字孪生。平台支持异构机器人形态、多模态视觉输入（如点云、体素）以及可定制的物理属性。

4.2 基线方法

实验使用以下五种基线方法评估 LLMHRL：

- LLMTeach [27]：一种由 LLM 指导的端到端强化学习方法，模型根据 LLM 给出的行为建议学习任务策略。它与 LLMHRL 同样使用 LLM 进行策略指导，因此作为主要对照。
- LLM：直接由 LLM 作出高层决策，再由低层策略模块执行具体动作。LLM 必须持续在线实时决策。通过比较可判断 LLMHRL 是否在训练早期有效学习 LLM 的高层策略，并在后期逐渐超越 LLM。
- Inner Monologue [18]：使用脚本模块，以物体识别、成功检测和任务进度描述等语言反馈帮助 LLM 复查高层决策。LLM 同样必须在线实时决策。该对照用于评估 LLMHRL 是否能超越 LLM 的思维链能力。
- Hierarchical RL [32]：采用基于选项的 HRL，每个选项对应一个与 LLMHRL 一致的低层策略。该比较用于判断 LLMHRL 是否有效学到了 LLM 的高层策略。
- RL (PPO) [33]：采用近端策略优化算法的端到端强化学习基线，用于展示典型 RL 算法在稀疏奖励环境中的表现。

4.3 评估指标

- (1) 成功率：智能体成功到达最终目标的回合比例，直接反映策略的可靠性和有效性。
- (2) 平均回报：每个回合获得的累计奖励均值。平均回报越高，通常表明策略越优。
- (3) 收敛速度：达到至少 80% 成功率所需的训练步数，用于衡量算法的样本效率和形成有效策略的速度。

4.4 大语言模型

本文通过 API 调用 ChatGPT-4 作为 LLM，指导 LLMHRL 的高层策略；其强大的推理能力能够提供实用的指导策略。为保证公平，对比实验中的 LLM 方法也使用同一模型。

为提高 LLM 的推理能力，本文使用思维链式文本提示，其中包含任务文本和环境文本。任务文本给出当前任务规则与具体示例，帮助 LLM 理解各高层决策的含义和输出格式；环境文本来自智能体观测，并由 GPT-4V 分析、解释。LLM 需要依据不同的环境观测文本给出适当的高层策略指导。输出内容包括对当前环境的解释、智能体下一步应采取的高层决策，以及对应决策的关键词。具体示例如图 4 所示。

图 4 任务提示示例（中文重排）

任务文本
在一个上锁的二维网格房间中，智能体需要打开门。规则：①只有持有与门颜色一致的钥匙并面向门时才能开门；②只有面向钥匙时才能拿起；③前往对象后即面向该对象；④一次只能持有一个物体；⑤可执行 explore、go to <object>、pick up <object>、drop <object>、open <object>。请说明智能体看到和持有什么，推断当前可做的事情，最后选择正确动作。
环境文本示例
观测：智能体看到 <red key>、<blue door>、<blue key>；持有 <nothing>；面向 <nothing>。输出动作：{go to <blue key>}。

4.5 MiniGrid 实验

本节在四个难度逐渐增加的 MiniGrid 任务上评估 LLMHRL，并与其他基线比较。结果显示，LLMHRL 始终获得更高的成功率、更好的平均回报和明显更快的训练收敛；在更复杂、传统 RL 完全失败的任务上优势尤为明显。消融实验进一步证实，将高层策略分解为独立的“技能”和“目标”，可通过缩小动作空间改善性能与探索效率；同时还分析了关键学习参数对成功率的影响。

4.5.1 实验设置

- (1) 任务配置：基于 MiniGrid 构建四个难度递增的任务，如图 5 所示。任务越难，决策链越长，而且为获得奖励并到达目标，对错误行为的容忍度越低。

Infor

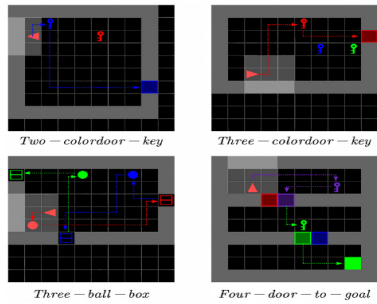


图 5 四类难度递增的任务示例。双色门钥匙和三色门钥匙是带干扰项的单钥匙-门匹配；三球三盒是多对象分拣；四门寻目标包含多层探索和钥匙-门匹配。虚线表示智能体使用不同技能与目标得到的最优轨迹。

- 双色门钥匙（简单）：生成两把钥匙和一扇门，智能体需找到与门颜色匹配的钥匙，拿起并开门。
- 三色门钥匙（中等）：生成三把钥匙和一扇门，智能体需找到正确颜色的钥匙并开门。
- 三球三盒（中等）：生成三种颜色的球和盒子，智能体需将每个球放入颜色匹配的盒子。
- 四门寻目标（困难）：环境含三层。前两层需探索并找到对应门的钥匙、拿起钥匙、开门并放下钥匙；同时存在没有对应钥匙的假门作为干扰。最后一层需找到并到达目标。

(2) 奖励函数：上述任务使用同一奖励函数：

$$r = (1 - \text{step}_{\text{finish}} / \text{step}_{\text{max}}) \cdot \text{done}$$

(8)

$\text{step}_{\text{finish}}$ 表示完成当前任务所用的低层策略步数， step_{max} 表示允许执行的最大低层步数；done 表示仅在任务完成后给出奖励。这一设计保证了稀疏奖励条件下仍能有效完成任务。

(3) 低层策略：MiniGrid 的低层策略库包含 5 种技能和 15 类目标。探索和前往使用 A* 寻路；探索随机选择附近空位置作为目的地，前往则以目标坐标为目的地。拿起、放下、打开直接使用 MiniGrid 提供的动作。

表 2 MiniGrid 技能及其对应的低层策略

技能	低层策略
explore（探索）	A*
go to（前往）	A*
pick up（拿起）	单步动作
put down（放下）	单步动作
open（打开）	单步动作

表 3 MiniGrid 任务可用的技能与目标

任务	可用技能	可用目标
双色门钥匙	全部	红/蓝钥匙；蓝门
三色门钥匙	全部	红/蓝/绿钥匙；蓝门
三球三盒	全部	红/蓝/绿球；红/蓝/绿盒
四门寻目标	全部	紫/绿钥匙；红/蓝/绿/紫门；目标

4.5.2 竞争性结果

表 4 和图 6 给出了五种方法在四个任务中的成功率。LLMHRL 在四个任务中的三个取得最高成功率，说明它能够有效适应不同任务。表 6 还表明，LLMHRL 的收敛速度明显快于其他方法，充分利用了训练早期的 LLM 指导并迅速改善任务表现。

表 4 四个任务的成功率

方法	双色门钥匙	三色门钥匙	三球三盒	四门寻目标
LLMHRL	0.928	0.883	0.936	0.887
LLMTeach	0.928	0.903	0.813	0.405
Inner Monologue	0.890	0.800	0.780	0.710
LLM	0.770	0.590	0.600	0.480
RL（Hierarchy）	0.216	0.048	0.000	0.002
RL（PPO）	0.064	0.017	0.000	0.000

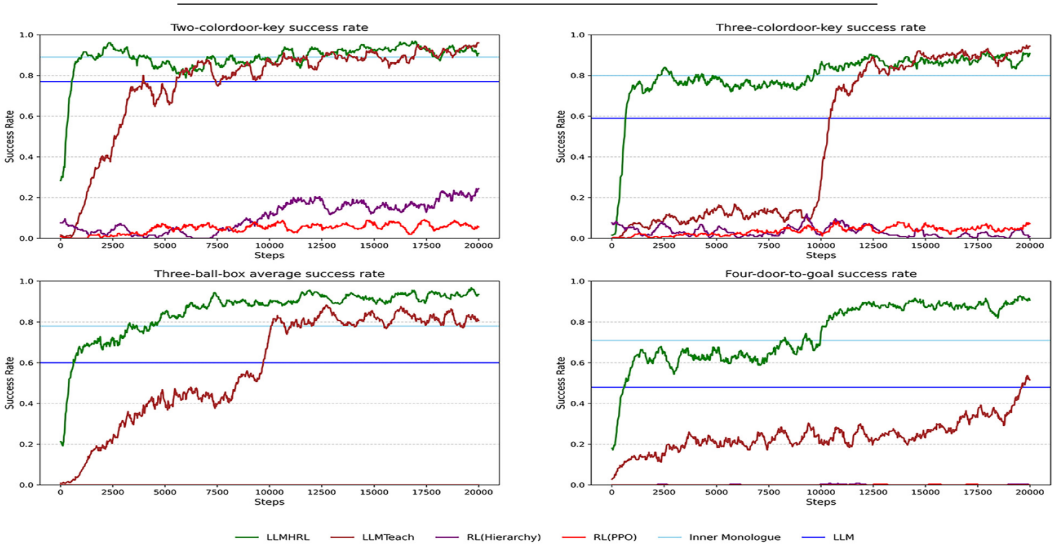


图 6 四个任务训练过程中的成功率。绿色：LLMHRL；棕红：LLMTeach；紫色：层级 RL；红色：PPO；浅蓝：Inner Monologue；蓝色：LLM。

表 5 四个任务的平均回报

方法	双色门钥匙	三色门钥匙	三球三盒	四门寻目标
LLMHRL	0.726	0.668	0.591	0.609
LLMTeach	0.759	0.667	0.528	0.243
Inner Monologue	0.700	0.600	0.450	0.370
LLM	0.600	0.520	0.320	0.190
RL (Hierarchy)	0.087	0.024	0.000	0.000
RL (PPO)	0.027	0.009	0.000	-0.001

表 6 四个任务的收敛速度（训练步数）

方法	双色门钥匙	三色门钥匙	三球三盒	四门寻目标
LLMHRL	1,250	2,000	5,000	11,500
LLMTeach	5,000	10,000	11,000	∞
Inner Monologue	0	0	∞	∞
LLM	∞	∞	∞	∞
RL (Hierarchy)	∞	∞	∞	∞
RL (PPO)	∞	∞	∞	∞

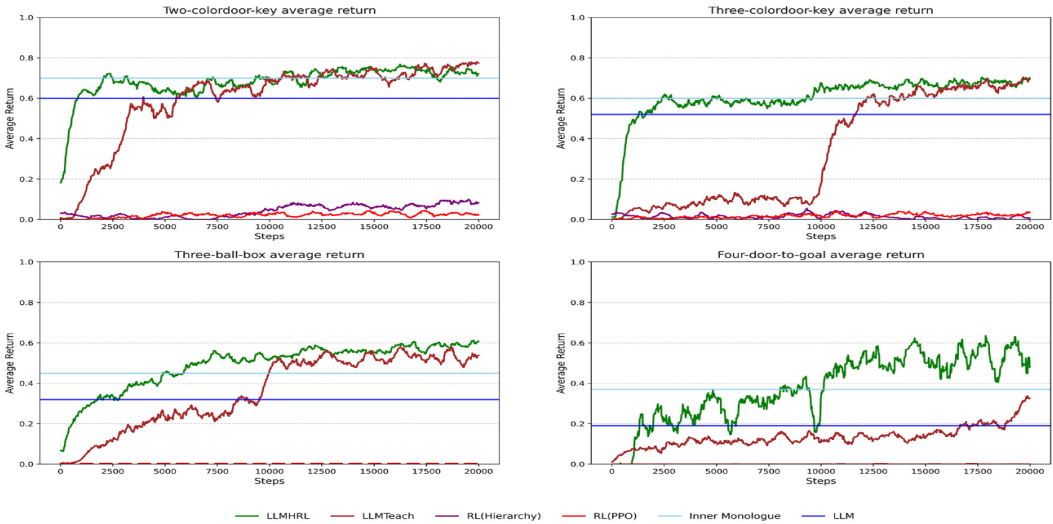


Fig. 7. Average return of four tasks during training.

图 7 四个任务训练过程中的平均回报。颜色含义与图 6 相同。

表 5 和图 7 展示了五种方法在四个任务中的平均回报。LLMHRL 在其中三个任务获得最高平均回报，且均超过 LLM 与 Inner Monologue。这说明教师-学生范式能够逐步学习策略，比单纯向 LLM 引入环境反馈更鲁棒。随着任务难度上升，LLMHRL 仍能保持较高的成功率与平均回报，而其他方法下降更明显。

LLMTeach 在多数任务中表现良好，并可在 10,000 步内收敛到有效策略，但面对更复杂的任务较为困难。原因是学生需要学习低层动作，既要掌握多种技能，又要以精确顺序执行；决策链越长，在 LLM 指导下学到正确策略所需的时间就越长。

常规 RL 无论是否采用分层结构都未能学到有效策略，仅缩短决策链仍不足以取得满意结果。实验表明，引入 LLM 对帮助智能体克服这些学习困难至关重要。

4.5.3 技能与目标分解的消融实验

LLMHRL 将高层策略分解为技能与目标的组合。为验证这一设计，本文将其与不进行分解、直接给出全部高层策略选项的方法比较，后者记为 LLMHRL (NS)。

表 7 消融实验的成功率与平均回报

成功率	双色门钥匙	三色门钥匙	三球三盒	四门寻目标
LLMHRL	0.928	0.883	0.936	0.887
LLMHRL (NS)	0.874	0.842	0.884	0.867

平均回报	双色门钥匙	三色门钥匙	三球三盒	四门寻目标
LLMHRL	0.726	0.681	0.591	0.518
LLMHRL (NS)	0.698	0.596	0.507	0.609

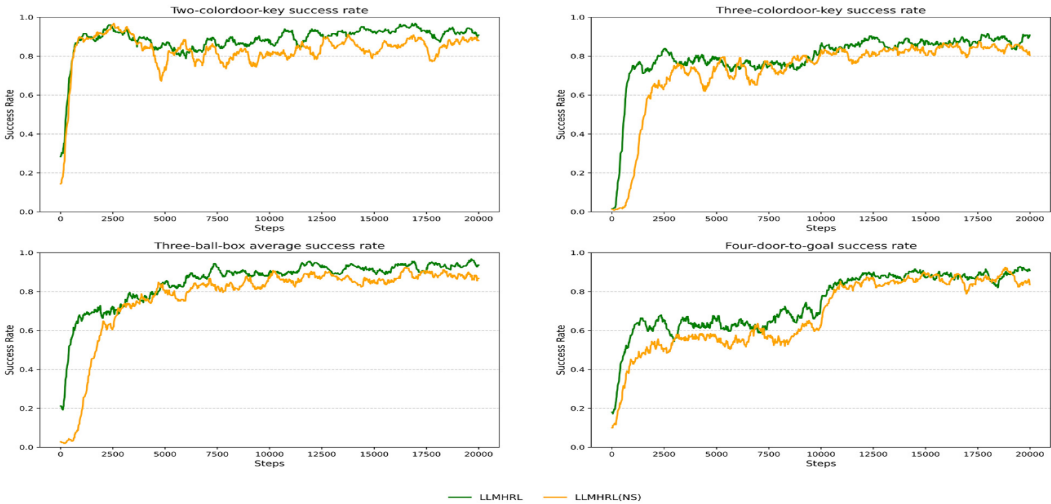


图 8 技能与目标分解消融实验：四个任务训练过程中的成功率。绿色为 LLMHRL，橙色为 LLMHRL (NS)。

如表 7、图 8 和图 9 所示，将动作空间拆分为技能与目标，能够提高四个任务的表现。困难任务会扩大动作空间并降低探索效率；将动作空间拆成两部分后，LLMHRL 能够有效缩小动作空间，使高层策略更好地学习 LLM 的指导策略。

4.5.4 学习参数 i_1 与 λ_0 的消融实验

LLMHRL 使用退火参数 λ 控制教师对学生的指导程度。 λ 从初值 λ_0 开始，在 i_1 个训练步内下降到 0。本节验证这种指导是否有效，并分析 λ_0 与终止步数 i_1 对训练的影响。

Q. Li, B. Pang, Y. Song et al.

Information Sciences 723 (2026) 122688

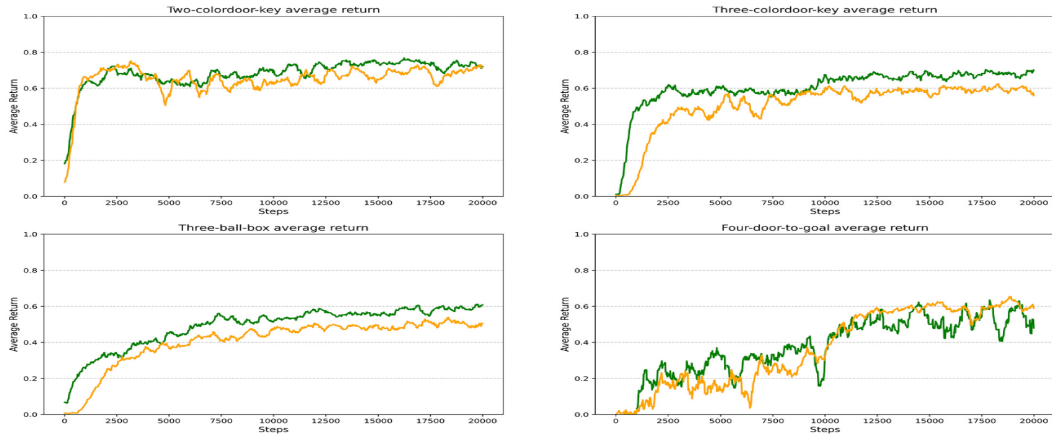


图 9 技能与目标分解消融实验：四个任务训练过程中的平均回报。绿色为 LLMHRL，橙色为 LLMHRL (NS)。

Fig. 9. Average return of four tasks during training of ablation study.

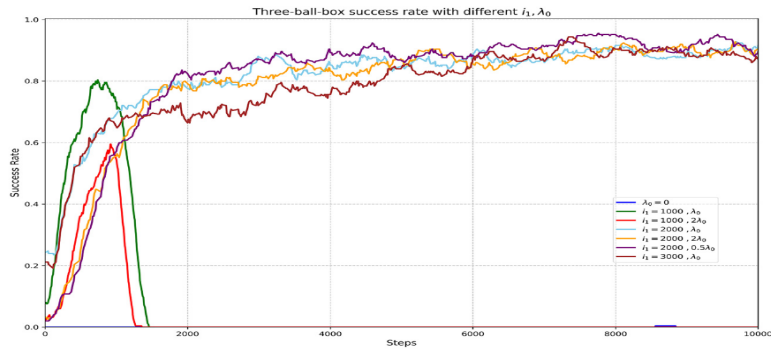


图 10 在不同 i_1 与 λ_0 设置下，三球三盒任务的成功率。

如图 10 所示， λ_0 会影响收敛速度。原式 (7) 通过按 1:1 平衡教师指导与环境奖励来计算 λ_0 ；将其增大到 $2\lambda_0$ 或减小到 $0.5\lambda_0$ 都会降低训练效率，而 $\lambda_0=0$ 、完全没有教师指导时无法促成有效训练。

终止步数 i_1 决定训练能否成功。若 i_1 太小（如 1,000 步即终止），训练无法成功；2,000 步最优，继续增加步数虽仍能成功，却会损害性能。 λ_0 增大或减小时，这一结论仍保持一致。

4.6 ManiSkill 实验

本节在 ManiSkill 平台设置三个难度递增的桌面操作任务：单立方体拾取、三立方体拾取和锤击钉子。机器人仅使用局部视觉观测；预配置转换器将 RGB 图像转换为自然语言，在不同的技能和目标选项下执行长序列决策。结果表明，LLMHRL 在所有任务上均取得最高成功率和平均回报，并展现出快速学习进展，克服了 LLM 难以整合执行反馈、RL 在稀疏奖励下学习困难等问题。

4.6.1 实验设置

(1) 任务配置：本文在 ManiSkill 中设计了三个机器人桌面操作任务，均属于难度逐渐增加的长序列决策任务。

单立方体拾取：桌面上放置位置和颜色随机的立方体，以及三个颜色不同的目标区域（图 11）。机器人需拿起立方体，并将其放到颜色匹配的目标上。

三立方体拾取：引入三个颜色不同、位置随机的立方体及对应目标区域（图 12）。机器人需依次拿起每个立方体，并放到相同颜色的目标上。由于必须连续操作多个物体，决策时域显著延长。

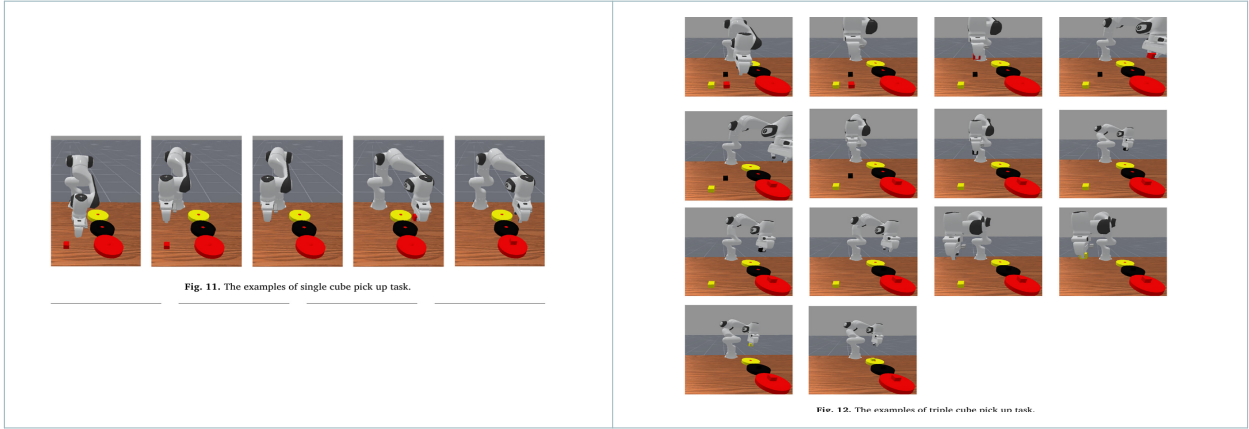


图 11（左）单立方体拾取任务示例；图 12（右）三立方体拾取任务示例。

Infor

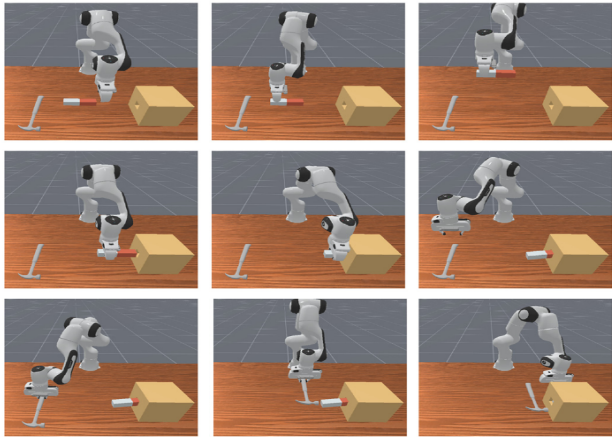


图 13 锤击钉子任务示例。

锤击钉子：该任务包含锤子、钉子和带孔立方体（图 13），挑战更大。机器人首先需要抓取钉子，使其与孔对齐并插入；随后抓取锤子，对准钉入位置并敲击完成任务。该设计引入复杂任务规则和严格的动作顺序要求，进一步提高了难度。

(2) 奖励函数：上述任务使用同一奖励函数：

$$r = [2 - 0.5 \cdot (\text{steps}_{\text{finish}} / \text{steps}_{\text{max}})] \cdot \text{done}$$

(9)

(3) 低层策略：在所有任务中，高层策略只接收安装在末端执行器上方的相机所采集的局部 RGB 视觉观测。本文采用六种低层策略选项：探索、前往、拿起、放置、插入和敲击，见表 8、表 9。

低层策略由 RRT 算法驱动。该算法从初始点 x_0 出发，经过目标点 x_i ，为机器人规划轨迹 τ 。对于每种技能与目标组合，目标点由人工预先标注，并从环境中提取。

表 8 ManiSkill 技能及其对应的低层策略

技能	低层策略
explore（探索）	RRT*
go to（前往）	RRT*
pick up（拿起）	闭合夹爪
place（放置）	张开夹爪
strike（敲击）	RRT*
insert（插入）	RRT*

表 9 ManiSkill 任务可用的技能与目标

任务	可用技能	可用目标
单立方体拾取	全部	红/黄/黑目标；红色立方体
三立方体拾取	全部	红/黄/黑目标；红/黄/黑立方体
锤击钉子	全部	锤子；钉子；孔

4.6.2 竞争性结果

图 14 和表 10 展示了四种方法在训练期间的成功率。在三个难度逐步增加的任务中，LLMHRL 的成功率分别达到 92%、75% 和 72%，始终优于单独使用 LLM。图 15 和表 11 展示了四种方法训练期间的平均回报，其性能趋势与成功率曲线一致。如表 12 所示，在 LLM 的指导下，LLMHRL 的成功率能够快速提升。

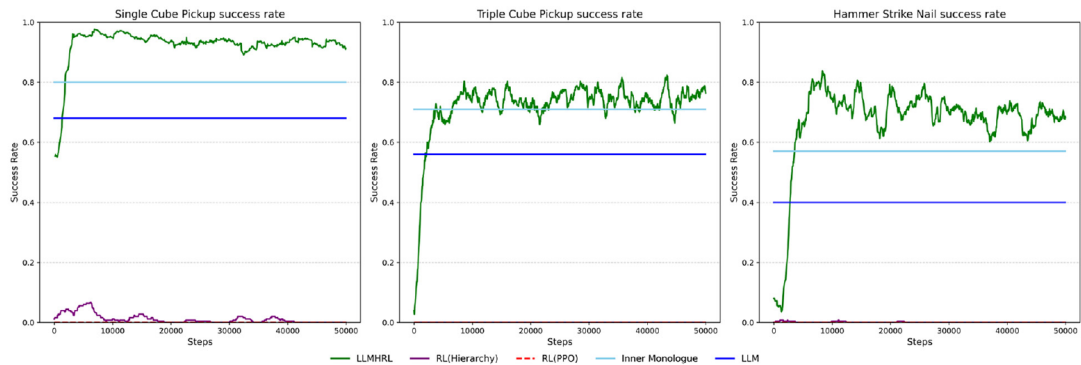


图 14 三个 ManiSkill 任务训练期间的成功率。原文图注写作“训练期间四个任务的成功率”，图中实际展示三个任务。

表 10 三个任务的成功率

方法	单立方体拾取	三立方体拾取	锤击钉子
LLMHRL	0.95	0.77	0.71
Inner Monologue	0.80	0.71	0.57
LLM	0.68	0.56	0.45
RL (Hierarchy)	0.01	0.00	0.01
RL (PPO)	0.00	0.00	0.00

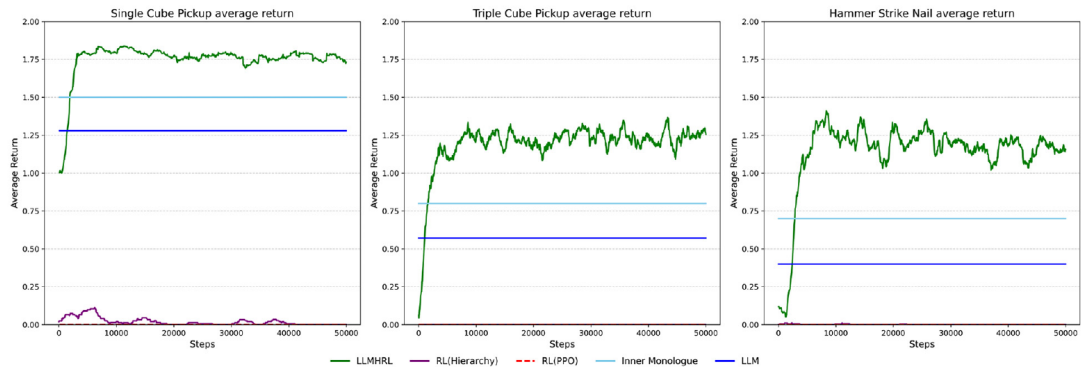


图 15 三个 ManiSkill 任务训练期间的平均回报。原文图注写作“训练期间四个任务的平均回报”，图中实际展示三个任务。

表 11 三个任务的平均回报

方法	单立方体拾取	三立方体拾取	锤击钉子
LLMHRL	1.76	1.25	1.16
Inner Monologue	1.50	0.80	0.70
LLM	1.28	0.57	0.40
RL (Hierarchy)	0.18	0.00	0.02
RL (PPO)	0.00	0.00	0.00

表 12 三个任务的收敛速度（训练步数）

方法	单立方体拾取	三立方体拾取	锤击钉子
LLMHRL	5,000	10,000	10,000
Inner Monologue	0	∞	∞
LLM	∞	∞	∞
RL (Hierarchy)	∞	∞	∞
RL (PPO)	∞	∞	∞

尽管 Inner Monologue 能够接收语言形式的环境反馈，但在部分场景中仍无法成功。这是因为 LLM 针对特定任务进行调整的能力有限。若没有此类反馈，单独使用 LLM 的表现又低于 Inner Monologue。与此同时，HRL 在训练早期偶尔能够完成任务，但稀疏奖励使其难以学习有效策略。PPO 直接输出低层动作，在缺乏密集奖励信号时，整个训练过程中的成功率始终为 0%。

4.7 物理机器人评估

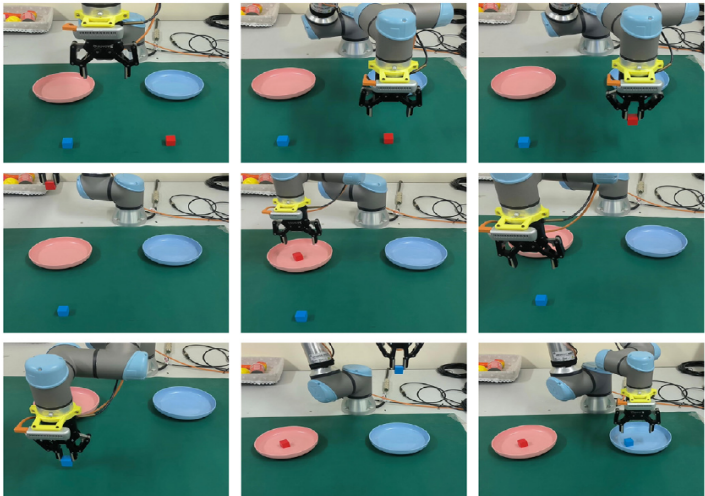


图 16 物理机器人测试过程。原文图注误写为“训练期间四个任务的平均回报”。

本文在物理机器人平台上评估 LLMHRL 的性能，如图 16 所示。评估平台采用六自由度 UR5 机械臂，并在末端执行器上安装 Robotiq-85 夹爪以提供抓取能力；末端还安装 Intel RealSense D435 RGB-D 相机，用于采集视觉输入。测试任务与 ManiSkill 环境中的“拾取立方体”任务相似，要求机器人将红色和蓝色立方体分别放入桌面上颜色对应的容器。

首先，本文在 ManiSkill 中配置与真实场景一致的仿真环境，并在其中训练 LLMHRL；随后将训练好的模型迁移到物理机器人进行测试。评估时，相机采集的 RGB 图像作为模型的状态输入，模型输出高层动作，再由预定义的机器人技能执行。本文在真实场景中进行了 20 轮测试，其中 16 轮成功完成任务。结果表明，LLMHRL 能够可靠地部署到真实环境中的物理机器人上。

4.8 LLM 失效分析

表 13 任务失效类别

任务	LLM 成功	观察抽象失败	LLM 格式失败	LLM 推理失败	低层执行失败
四门到目标 (MiniGrid)	48%	15%	10%	27%	0%
锤击钉子 (ManiSkill)	45%	10%	5%	35%	5%
拾取立方体 (真实机器人)	40%	20%	2%	18%	20%

为了研究将 LLM 作为零样本高层策略时存在的不足，本文进行了详细的失效分析。人工评审者将失败划分为四类。如表 13 所示，LLM 的总体成功率仍然有限，主要原因是其缺乏任务特定的内在知识，而且输出可能不一致。

观察抽象失败：使用 VLM 将环境状态转换为文本描述时，所得表述可能含糊或令 LLM 困惑。例如，输入“看到红色立方体和绿色立方体；手中拿着绿色立方体”时，LLM 可能仍然输出“拿起绿色立方体”，造成重复动作。

LLM 格式失败：LLM 的输出不符合低层模块可执行的预期格式。例如，当要求输出预定义动作时，模型可能生成无效命令“exploit”，而不是有效命令“explore”，从而导致解析错误。

LLM 推理失败：LLM 未能理解环境逻辑或任务目标。例如，面对“红色钥匙”和“蓝色门”时，模型可能因简单的对象联想偏差而直接输出“拿起红色钥匙”，而没有推断出应当寻找蓝色钥匙；正确动作应是继续“探索”合适的钥匙。

低层执行失败：先前动作可能使后续命令在物理上无法执行。例如，LLM 先要求机器人将一个立方体放到另一个立方体上，随后又要求拿起底部立方体，此时机器人可能无法完成该动作，最终导致任务失败。

不同任务和环境中的失效类型分布差异明显。四门到目标任务需要频繁探索，命令数量较多，产生错误格式输出的机会也更多，因此 LLM 格式失败更常见。锤击钉子任务要求理解“先放置钉子、再敲击”的程序依赖关系，主要触发 LLM 推理失败。可以预期，在不可预测物理交互更常见的复杂真实环境中，低层执行失败会显著增加。

4.9 计算成本与硬件配置

表 14 单次环境交互的时延

方法	训练阶段	部署阶段
LLMHRL	250 ms	<10 ms
LLMTeach	5,000 ms	<10 ms
Inner Monologue	无需训练	25,000 ms
LLM	无需训练	5,000 ms
RL (Hierarchy)	<10 ms	<10 ms

所有实验（包括训练和部署）均在单张 NVIDIA GeForce RTX 3090 GPU 上完成。为评估各方法的实际适用性，本文测量了表 14 所示的推理时延。这里的时延定义为智能体完成一次高层动作所需的平均时间。

LLMHRL：训练阶段，LLM 每推荐一次高层动作，低层策略可连续执行多个动作步骤，从而避免为每一个低层动作查询 LLM 所带来的时延。部署阶段仅依赖轻量级学生策略，因此时延可忽略不计。

LLMTeach：训练阶段，每一个学生低层动作都需要调用一次 LLM 以获得教师指导；但与 LLMHRL 相同，部署阶段只使用训练好的学生网络，从而避免 LLM 带来的时延。

Inner Monologue 与 LLM：部署时，Inner Monologue 需要多次调用 LLM，根据环境反馈进行迭代式自我修正，而 LLM 方法只调用一次。依赖在线 LLM 推理会引入显著时延，严重妨碍其在实时场景中的实际应用。

综上，LLMHRL 在性能和时延之间取得了有效平衡。它将 LLM 推理限制在训练阶段，使部署过程能够以极低时延快速响应。

5. 结论

本文将基于选项的分层强化学习与 LLM 结合，以应对稀疏奖励环境中的长序列决策难题。LLMHRL 计算 LLM 与 RL 高层策略动作分布之间的差异，并将其加入损失函数，以此指导 RL。高层动作被拆分为技能与目标两个组成部分，从而减少可选动作数量；低层动作则由技能库构成，以支持不同的高层决策。实验表明，LLMHRL 能够更快地学习 LLM 的策略，并通过与环境交互逐步习得更优策略。此外，LLMHRL 可部署到物理机器人，并在真实应用中表现出较强的鲁棒性。其局限在于高层动作选项固定，因此难以处理未知目标。未来工作将继续改进 LLMHRL。

CRediT 作者贡献声明

Qianxi Li：撰写初稿、软件、方法论、数据整理、概念构建。Bao Pang：审阅与编辑、监督、项目管理。Yong Song：监督、资源、经费获取。Hongze Fu：撰写初稿、软件、方法论、数据整理。Qingyang Xu：监督、资源、经费获取。Xianfeng Yuan：监督、资源、经费获取。Xiaolong Xu：监督、资源、经费获取。Chengjin Zhang：监督、资源、经费获取。

利益冲突声明

作者声明，不存在可能影响本文所报告工作的已知竞争性经济利益或个人关系。

致谢

本研究得到国家自然科学基金资助（项目编号：62203266）。

数据可用性

相关数据可根据请求提供。

参考文献

为保证检索准确性，参考文献中的作者、英文题名、期刊或会议名称、卷页信息及 DOI 按原文保留。

[1] B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, et al., Language models are few-shot learners, arXiv preprint arXiv:2005.14165, 2020, <https://doi.org/10.5555/3495724.3495883>.

[2] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, S. Altman, S. Anadkat, et al., GPT-4 technical report, arXiv preprint arXiv:2303.08774, 2023, <https://doi.org/10.48550/arXiv.2303.08774>.

[3] R. T. Icarte, T. Q. Klassen, R. Valenzano, S. A. McIlraith, Reward machines: exploiting reward function structure in reinforcement learning, J. Artif. Intell. Res. 73 (2022) 173–208, <https://doi.org/10.1613/jair.1.12440>.

[4] X. Liu, G. Wang, Z. Liu, Y. Liu, Z. Liu, P. Huang, Hierarchical reinforcement learning integrating with human knowledge for practical robot skill learning in complex multi-stage manipulation, IEEE Trans. Autom. Sci. Eng. 21 (3) (2024) 3852–3862, <https://doi.org/10.1109/TASE.2023.3288037>.

[5] D. Driess, F. Xia, M. S.M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, et al., PaLM-E: an embodied multimodal language model, in: Proceedings of the 40th International Conference on Machine Learning, PMLR, 2023, pp. 8469–8488, <https://doi.org/10.1016/j.cag.2024.01.012>.

[6] H. Kheddar, Transformers and large language models for efficient intrusion detection systems: a comprehensive survey, Inf. Fusion (2025) 103347.

[7] L. Chen, Y. Lei, S. Jin, Y. Zhang, L. Zhang, RLingua: improving reinforcement learning sample efficiency in robotic manipulations with large language models, IEEE Robot. Autom. Lett. 9 (7) (2024) 6075–6082, <https://doi.org/10.1109/LRA.2024.3400189>.

[8] A. K. Lampinen, I. Dasgupta, S. C.Y. Chan, H. R. Sheahan, A. Creswell, D. Kumaran, J. L. McClelland, F. Hill, Language models, like humans, show content effects on reasoning tasks, PNAS Nexus 3 (7) (2024) pgae233, <https://doi.org/10.1093/pnasnexus/pgae233>.

[9] R. Wang, D. Zhao, Z. Yuan, I. Obi, B.-C. Min, PrefCLM: enhancing preference-based reinforcement learning with crowdsourced large language models, IEEE Robot. Autom. Lett. 10 (3) (2025) 2486–2493, <https://doi.org/10.1109/LRA.2025.3528663>.

- [10] B. Yu, H. Kasaei, M. Cao, L3MVN: leveraging large language models for visual target navigation, in: 2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), IEEE, 2023, pp. 3554–3560, <https://doi.org/10.1109/IROS55552.2023.10342512>.
- [11] F. Bang, GPTCache: an open-source semantic cache for LLM applications enabling faster answers and cost savings, in: Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023), 2023, pp. 212–218, <https://doi.org/10.18653/v1/2023.nlpss-1.24>.
- [12] F. Yang, J. Liu, K. Li, LLM-guided reinforcement learning for interactive environments, *Mathematics* 13 (12) (2025) 1932, <https://doi.org/10.3390/math13121932>.
- [13] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, Y. Su, LLM-Planner: few-shot grounded planning for embodied agents with large language models, in: Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023, pp. 2998–3009, <https://doi.org/10.1109/ICCV51070.2023.00280>.
- [14] S. Sun, R. Liu, J. Lyu, J.-W. Yang, L. Zhang, X. Li, A large language model-driven reward design framework via dynamic feedback for reinforcement learning, *Knowl.-Based Syst.* 326 (2025) 114065, <https://doi.org/10.1016/j.knosys.2025.114065>.
- [15] R. Miao, Q. Jia, F. Sun, G. Chen, H. Huang, Hierarchical understanding in robotic manipulation: a knowledge-based framework, in: *Actuators*, vol. 13, MDPI, 2024, p. 28, <https://doi.org/10.3390/act13010028>.
- [16] S. Hao, Y. Gu, H. Ma, J. J. Hong, Z. Wang, D. Z. Wang, Z. Hu, Reasoning with language model is planning with world model, in: The 2023 Conference on Empirical Methods in Natural Language Processing, 2023, <https://doi.org/10.18653/v1/2023.emnlp-main.507>.
- [17] K. Valmeekam, M. Marquez, A. Olmo, S. Sreedharan, S. Kambhampati, PlanBench: an extensible benchmark for evaluating large language models on planning and reasoning about change, *Adv. Neural Inf. Process. Syst.* 35 (2022) 38975–38987, <https://doi.org/10.5555/3666122.3667815>.
- [18] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, et al., Inner Monologue: embodied reasoning through planning with language models, in: 6th Annual Conference on Robot Learning, 2022.
- [19] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al., Chain-of-thought prompting elicits reasoning in large language models, *Adv. Neural Inf. Process. Syst.* 35 (2022) 24824–24837, <https://doi.org/10.5555/3600270.3602070>.
- [20] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, Y. Iwasawa, Large language models are zero-shot reasoners, *Adv. Neural Inf. Process. Syst.* 35 (2022) 22199–22213, <https://doi.org/10.5555/3600270.3601883>.
- [21] S. Chakraborty, J. Qiu, H. Yuan, A. Koppel, F. Huang, D. Manocha, A. Bedi, M. Wang, MaxMin-RLHF: towards equitable alignment of large language models with diverse human preferences, in: ICML 2024 Workshop on Models of Human Feedback for AI Alignment, 2024.
- [22] W. Tan, W. Zhang, S. Liu, L. Zheng, X. Wang, B. An, True knowledge comes from practice: aligning LLMs with embodied environments via reinforcement learning, in: The Twelfth International Conference on Learning Representations, 2024.
- [23] T. Carta, C. Romac, T. Wolf, S. Lamprier, O. Sigaud, P.-Y. Oudeyer, Grounding large language models in interactive environments with online reinforcement learning, in: International Conference on Machine Learning, PMLR, 2023, pp. 3676–3713, <https://doi.org/10.5555/3618408.3618558>.
- [24] A. Szot, M. Schwarzer, H. Agrawal, B. Mazouze, R. Metcalf, W. Talbott, N. Mackraz, R. D. Hjelm, A. T. Toshev, Large language models as generalizable policies for embodied tasks, in: The Twelfth International Conference on Learning Representations, 2023.
- [25] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, A. Anandkumar, Eureka: human-level reward design via coding large language models, in: The Twelfth International Conference on Learning Representations, 2024.
- [26] T. Xie, S. Zhao, C. H. Wu, Y. Liu, Q. Luo, V. Zhong, Y. Yang, T. Yu, Text2Reward: reward shaping with language models for reinforcement learning, in: The Twelfth International Conference on Learning Representations, 2024.
- [27] Z. Zhou, B. Hu, C. Zhao, P. Zhang, B. Liu, Large language model as a policy teacher for training reinforcement learning agents, *Proc. Int. Jt. Conf. Artif. Intell.* (2024) 5671–5679.
- [28] Z. Shen, T. Zhu, Q. Sun, S. Gao, J. Li, Beyond human preferences: exploring reinforcement learning trajectory evaluation and improvement through LLMs, *Proc. Int. Jt. Conf. Artif. Intell.* (2024).
- [29] A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, et al., Do as I can, not as I say: grounding language in robotic affordances, in: Conference on Robot Learning, PMLR, 2023, pp. 287–318.
- [30] R. Hazra, P. Z. Dos Martires, L. De Raedt, SayCanPay: heuristic planning with large language models using learnable domain knowledge, in: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 38, 2024, pp. 20123–20133.
- [31] Z. Hu, W. Liu, X. Qu, X. Yue, C. Chen, Z. Wang, Y. Cheng, Divide and conquer: grounding LLMs as efficient decision-making agents via offline hierarchical reinforcement learning, Forty-second International Conference on Machine Learning, 2025.
- [32] M. Matthews, M. Samvelyan, J. Parker-Holder, E. Grefenstette, T. Rocktäschel, Hierarchical kickstarting for skill transfer in reinforcement learning, *Proc. Mach. Learn. Res.* (2022) 856–874.
- [33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, *arXiv preprint arXiv:1707.06347*, 2017.

作者简介



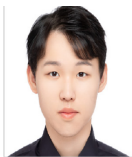
Qianxi Li 2023 年于上海理工大学机械工程学院获得机器人工程学士学位，目前在山东大学（威海）机电与信息工程学院攻读控制科学与工程硕士学位。研究方向包括深度强化学习、大语言模型和智能机器人控制。



Bao Pang 2014 年于辽宁科技大学获得运筹学与控制论硕士学位，2020 年于山东大学获得控制理论与控制工程博士学位。现任山东大学（威海）机电与信息工程学院讲师。研究方向包括智能机器人控制、多机器人协同控制、深度强化学习和群体智能。



Yong Song (IEEE 会员) 2001 年于山东大学（威海）获得控制科学学士学位，2008 年和 2012 年于山东大学分别获得模式识别与智能系统硕士、博士学位。现任山东大学机电与信息工程学院教授。研究方向包括机器人控制、机器学习和群体智能机器人。



Hongze Fu 现为山东大学（威海）机电与信息工程学院机器人工程专业大三本科生。研究方向包括通用机器人操作、深度强化学习和机器人基础模型。



Qingyang Xu 2007 年和 2010 年于大连海事大学分别获得控制理论与工程硕士、博士学位。2010 年 8 月至 2012 年 7 月在中国科学院大连化学物理研究所从事博士后研究。现任山东大学（威海）机电与信息工程学院副教授。研究方向包括人工智能、深度学习及其机器人应用。



Xianfeng Yuan 2017 年于山东大学获得控制理论与控制工程博士学位。2016 至 2017 年在美国俄克拉荷马州立大学联合培养。现任山东大学（威海）机电与信息工程学院副教授。研究方向包括机器学习、智能故障诊断和机器人技术。



Xiaolong Xu 出生于中国青岛。2011 年于青岛科技大学获得机械工程及自动化学士学位，2014 年于北京化工大学获得机械设计及理论硕士学位，2020 年于山东大学获得模式识别与智能系统博士学位。现任山东大学（威海）机电与信息工程学院助理教授。研究方向包括机器人、嵌入式系统，以及用于人机协作的可穿戴设备。



Chengjin Zhang 1997 年于东北大学获得工业自动化博士学位。研究方向包括控制理论及应用、智能机器人控制和生物信息学。